SOFTWARE ENGINEERING LAB-3

Component Modelling & Architectural Pattern Selection

SRN: PES1UG25CS847

Project: Community Solar Credit Allocation Manager

Domain: Sustainability & Green Tech

1. Problem Statement Continuity

This Lab-3 submission continues the same problem statement used in Lab-1: **Community Solar Credit Allocation Manager**. The system manages participating prosumer residents, shared solar-generation data, resident participation shares, monthly credit calculation and allocation, credit statements/history, and allocation disputes. The main actors remain the **Prosumer Resident** and **Co-op Manager**.

The component model below is derived from the Lab-1 functional requirements: resident registration and participation management, generation recording, credit calculation/allocation, resident statements, dispute handling, and secure auditable storage.

2. Component Diagram

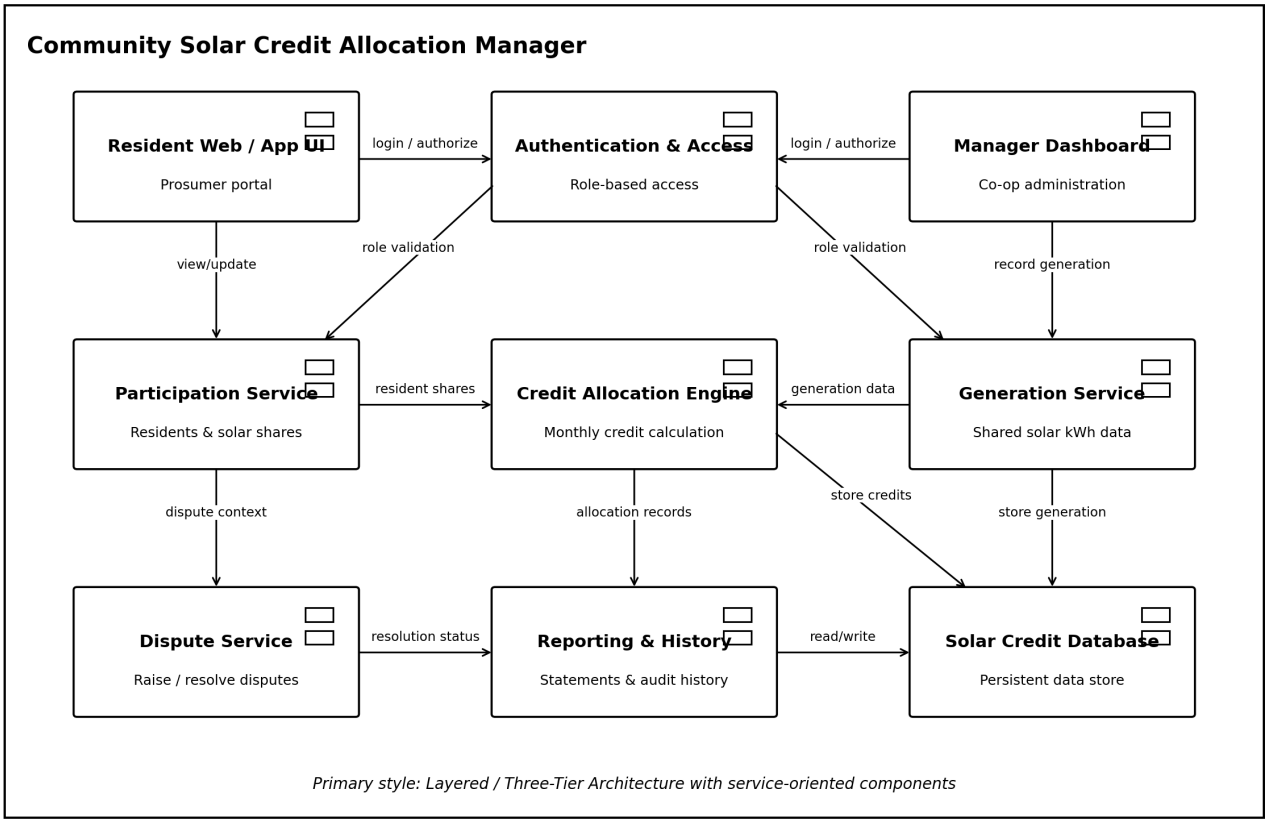


Figure 1. Component diagram for the Community Solar Credit Allocation Manager.

3. Component Responsibilities

Component	Responsibility
Resident Web / App UI	Allows a prosumer resident to view solar production, monthly credit statements and allocation history, and to raise a credit dispute.
Manager Dashboard	Allows the co-op manager to record generation data, manage resident participation/share information, run monthly allocations, and review disputes.

Component	Responsibility
Authentication & Access	Authenticates users and enforces role-based access so residents can access their own information while manager operations remain restricted.
Participation Service	Maintains participating residents, active/inactive status, and the allocation share used during monthly credit calculation.
Generation Service	Validates and stores the shared solar generation (kWh) for each billing period.
Credit Allocation Engine	Combines valid generation data and resident shares, calculates proportional credits, validates the allocation, and produces monthly allocation records.
Dispute Service	Records resident disputes, links them to the relevant billing period/allocation, and tracks manager remarks and resolution status.
Reporting & History	Provides resident credit statements, allocation history, and auditable views of previous processing results.
Solar Credit Database	Persists resident, participation, generation, allocation, dispute, user-access and audit information.

4. Architectural Pattern Selection

Selected Pattern: Layered / Three-Tier Architecture

The proposed system is organized into three logical layers: **Presentation Layer** (Resident UI and Manager Dashboard), **Application/Business Layer** (authentication, participation, generation, allocation, dispute, and reporting services), and **Data Layer** (Solar Credit Database). The business layer is further divided into cohesive components so that each major Lab-1 responsibility has a clear owner.

5. Architectural Justification

Separation of concerns: User-interface logic, solar-credit business rules, and persistent storage are separated. Changes to a resident screen do not require rewriting the allocation calculation or database logic.

Traceability to Lab-1 requirements: FR-001 and FR-006 map to Participation Service; FR-002 maps to Generation Service; FR-003 and FR-004 map to Credit Allocation Engine; FR-005 maps to Reporting & History; FR-007 maps to Dispute Service; security/audit requirements are handled by Authentication & Access and persistent audit data.

Maintainability: The allocation formula or participation rules can be modified in the business layer without redesigning the user interfaces. This is useful when the co-op changes its approved allocation policy.

Security: Role-based access is centralized. Prosumer residents are limited to their own credit information, while the co-op manager receives administrative functions. This supports the Lab-1 access-control requirement.

Reliability and auditability: Generation inputs, allocation results, disputes and manager actions can be stored as traceable records. This supports verification of a resident's monthly credit and investigation of disputes.

Scalability and performance: The allocation engine is isolated from presentation logic and can be optimized independently. For the Lab-1 requirement that a normal lab-scale monthly allocation complete within five seconds, calculation and database operations can be tuned without changing the UI.

Testability: Individual services can be unit-tested separately. For example, the Credit Allocation Engine can be tested with known generation and participation-share inputs to confirm that the calculated credits equal the distributable community credit.

Future extensibility: Additional components such as smart-meter integration, utility-billing integration, notifications, or automated solar-data import can later be added around the business layer without replacing the overall architecture.

6. Main Component Interaction

1. The Co-op Manager records or confirms shared solar generation for a billing period through the Manager Dashboard.
2. The Generation Service validates the kWh data and stores it.
3. The Credit Allocation Engine obtains active residents and their valid shares from the Participation Service.
4. The engine calculates each resident's proportional credit and stores the allocation result.
5. Reporting & History exposes the completed statement to the Prosumer Resident.
6. If a resident questions an allocation, the Dispute Service records the dispute for manager review and resolution.

7. Conclusion

The Layered / Three-Tier architectural pattern is appropriate for the Community Solar Credit Allocation Manager because it gives a clear separation between user interaction, credit-allocation business rules, and persistent data. The component diagram directly reflects the functions identified in Lab-1 and provides a modular structure that is maintainable, secure, testable and suitable for later integration with smart meters or utility systems.

8. Submission Note

Submit this PDF as the Lab-3 deliverable and upload it to the **same GitHub repository/link used for Lab-1**, as required by the lab instructions.

GitRepo Link: _____